

Link-Based vs. Array-Based Sequences

We close this chapter by reflecting on the relative pros and cons of array-based and link-based data **structures** that have been introduced thus far. The dichotomy between these approaches presents a common design decision when choosing an appropriate implementation of a data **structure**. There is not a one-size-fits-all solution, as each offers distinct advantages and disadvantages.

Advantages of Array-Based Sequences

- Arrays provide $O(1)$ -time access to an element based on an integer index. The ability to access the k th element for any k in $O(1)$ time is a hallmark advantage of arrays (see Section 5.2). In contrast, locating the k th element in a **linked** list requires $O(k)$ time to traverse the list from the beginning, or possibly $O(n \cdot k)$ time, if traversing backward from the end of a doubly **linked** list.
- Operations with equivalent asymptotic bounds typically run a constant factor more efficiently with an array-based **structure** versus a **linked structure**. As an example, consider the typical enqueue operation for a queue. Ignoring the issue of resizing an array, this operation for the `ArrayQueue` class (see Code Fragment 6.7) involves an arithmetic calculation of the new index, an increment of an integer, and storing a reference to the element in the array. In contrast, the process for a **LinkedList** (see Code Fragment 7.8) requires the instantiation of a node, appropriate linking of nodes, and an increment of an integer. While this operation completes in $O(1)$ time in either model, the actual number of CPU operations will be more in the **linked** version, especially given the instantiation of the new node.
- Array-based representations typically use proportionally less memory than **linked structures**. This advantage may seem counterintuitive, especially given that the length of a dynamic array may be longer than the number of elements that it stores. Both array-based lists and **linked** lists are referential **structures**, so the primary memory for storing the actual objects that are elements is the same for either **structure**. What differs is the auxiliary amounts of memory that are used by the two **structures**. For an array-based container of n elements, a typical worst case may be that a recently resized dynamic array has allocated memory for $2n$ object references. With **linked** lists, memory must be devoted not only to store a reference to each contained object, but also explicit references that link the nodes. So a singly **linked** list of length n already requires $2n$ references (an element reference and next reference for each node). With a doubly **linked** list, there are $3n$ references.

7.3. Doubly Linked Lists

273

7.3.1

Basic Implementation of a Doubly Linked List

We begin by providing a preliminary implementation of a doubly linked list, in the form of a class named `DoublyLinkedListBase`. We intentionally name the class with a leading underscore because we do not intend for it to provide a coherent public interface for general use. We will see that linked lists can support general insertions and deletions in $O(1)$ worst-case time, but only if the location of an operation can be succinctly identified. With array-based sequences, an integer index was a convenient means for describing a position within a sequence. However, an index is not convenient for linked lists as there is no efficient way to find the j th element; it would seem to require a traversal of a portion of the list.

When working with a linked list, the most direct way to describe the location of an operation is by identifying a relevant node of the list. However, we prefer to encapsulate the inner workings of our data structure to avoid having users directly access nodes of a list. In the remainder of this chapter, we will develop two public classes that inherit from our `DoublyLinkedListBase` class to provide more coherent abstractions. Specifically, in Section 7.3.2, we provide a `LinkedListDeque` class that implements the double-ended queue ADT introduced in Section 6.3; that class only supports operations at the ends of the queue, so there is no need for a user to identify an interior position within the list. In Section 7.4, we introduce a new `PositionalList` abstraction that provides a public interface that allows arbitrary insertions and deletions from a list.

Our low-level `DoublyLinkedListBase` class relies on the use of a nonpublic `Node` class that is similar to that for a singly linked list, as given in Code Fragment 7.4, except that the doubly linked version includes a `prev` attribute, in addition to the `next` and `element` attributes, as shown in Code Fragment 7.11.

class `Node`:

...Lightweight, nonpublic class for storing a doubly linked node....

slots

=

`_element` ,

`_prev` ,

`_next`

streamline memory

def

`init`

(`self`, `element`, `prev`, `next`):

initialize node's fields

`self.element` = `element`

user's element

`self.prev` = `prev`

previous node reference

`self.next` = `next`

next node reference

Code Fragment 7.11: A Python `Node` class for use in a doubly linked list.

The remainder of our `DoublyLinkedListBase` class is given in Code Fragment 7.12.

The constructor instantiates the two sentinel nodes and links them directly to each

Creativity

C-7.24 Give a complete implementation of the stack ADT using a singly **linked** list that includes a header sentinel.

C-7.25 Give a complete implementation of the queue ADT using a singly **linked** list that includes a header sentinel.

C-7.26 Implement a method, `concatenate(Q2)` for the **LinkedQueue** class that takes all elements of **LinkedQueue** Q2 and appends them to the end of the original queue. The operation should run in $O(1)$ time and should result in Q2 being an empty queue.

C-7.27 Give a recursive implementation of a singly **linked** list class, such that an instance of a nonempty list stores its `·rst` element and a reference to a list of remaining elements.

C-7.28 Describe a fast recursive algorithm for reversing a singly **linked** list.

C-7.29 Describe in detail an algorithm for reversing a singly **linked** list L using only a constant amount of additional space and not using any recursion.

C-7.30 Exercise P-6.35 describes a `LeakyStack` abstraction. Implement that ADT using a singly **linked** list for storage.

C-7.31 Design a forward list ADT that abstracts the operations on a singly **linked** list, much as the positional list ADT abstracts the use of a doubly **linked** list. Implement a `ForwardList` class that supports such an ADT.

C-7.32 Design a circular positional list ADT that abstracts a circularly **linked** list in the same way that the positional list ADT abstracts a doubly **linked** list, with a notion of a designated `·cursor·` position within the list.

C-7.33 Modify the **DoublyLinkedBase** class to include a `reverse` method that reverses the order of the list, yet without creating or destroying any nodes.

C-7.34 Modify the `PositionalList` class to support a method `swap(p, q)` that causes the underlying nodes referenced by positions p and q to be exchanged for each other. Relink the existing nodes; do not create any new nodes.

C-7.35 To implement the `iter` method of the `PositionalList` class, we relied on the convenience of Python's generator syntax and the `yield` statement. Give an alternative implementation of `iter` by designing a nested iterator class. (See Section 2.3.4 for discussion of iterators.)

C-7.36 Give a complete implementation of the positional list ADT using a doubly **linked** list that does not include any sentinel nodes.

C-7.37 Implement a function that accepts a `PositionalList` L of n integers sorted in nondecreasing order, and another value V, and determines in $O(n)$ time if there are two elements of L that sum precisely to V. The function should return a pair of positions of such elements, if found, or `None` otherwise.

8.3. Implementing Trees

327

8.3.3

Linked Structure for General Trees

When representing a binary tree with a linked structure, each node explicitly maintains fields left and right as references to individual children. For a general tree, there is no a priori limit on the number of children that a node may have. A natural way to realize a general tree T as a linked structure is to have each node store a single container of references to its children. For example, a children field of a node can be a Python list of references to the children of the node (if any). Such a linked representation is schematically illustrated in Figure 8.14.

element

parent

children

Baltimore

Chicago

New York

Providence

Seattle

(a)

(b)

Figure 8.14: The linked structure for a general tree: (a) the structure of a node; (b) a larger portion of the data structure associated with a node and its children.

Table 8.2 summarizes the performance of the implementation of a general tree using a linked structure. The analysis is left as an exercise (R-8.14), but we note that, by using a collection to store the children of each position p, we can implement children(p) by simply iterating that collection.

Operation

Running Time

len, is empty

O(1)

root, parent, is root, is leaf

O(1)

children(p)

O(cp + 1)

depth(p)

O(dp + 1)

height

O(n)

Table 8.2: Running times of the accessor methods of an n-node general tree implemented with a linked structure. We let cp denote the number of children of a position p. The space usage is O(n).

In Section 6.2.2, we introduced the notion of a circular array and demonstrated its use in implementing the queue ADT. In reality, the notion of a circular array was artificial, in that there was nothing about the representation of the array itself that was circular in structure. It was our use of modular arithmetic when advancing an index from the last slot to the first slot that provided such an abstraction.

In the case of linked lists, there is a more tangible notion of a circularly linked list, as we can have the tail of the list use its next reference to point back to the head of the list, as shown in Figure 7.7. We call such a structure a circularly linked list.

BOS

head

tail

LAX

MSP

ATL

Figure 7.7: Example of a singly linked list with circular structure.

A circularly linked list provides a more general model than a standard linked list for data sets that are cyclic, that is, which do not have any particular notion of a beginning and end. Figure 7.8 provides a more symmetric illustration of the same circular list structure as Figure 7.7.

ATL

BOS

current

MSP

LAX

Figure 7.8: Example of a circular linked list, with current denoting a reference to a select node.

A circular view similar to Figure 7.8 could be used, for example, to describe the order of train stops in the Chicago loop, or the order in which players take turns during a game. Even though a circularly linked list has no beginning or end, per se, we must maintain a reference to a particular node in order to make use of the list. We use the identifier current to describe such a designated node. By setting `current = current.next`, we can effectively advance through the nodes of the list.

Chapter 7. **Linked** Lists

In Chapter 5 we carefully examined Python's array-based list class, and in Chapter 6 we demonstrated use of that class in implementing the classic stack, queue, and dequeue ADTs. Python's list class is highly optimized, and often a great choice for storage. With that said, there are some notable disadvantages:

1. The length of a dynamic array might be longer than the actual number of elements that it stores.
2. Amortized bounds for operations may be unacceptable in real-time systems.
3. Insertions and deletions at interior positions of an array are expensive.

In this chapter, we introduce a data **structure** known as a **linked** list, which provides an alternative to an array-based sequence (such as a Python list). Both array-based sequences and **linked** lists keep elements in a certain order, but using a very different style. An array provides the more centralized representation, with one large chunk of memory capable of accommodating references to many elements. A **linked** list, in contrast, relies on a more distributed representation in which a lightweight object, known as a node, is allocated for each element. Each node maintains a reference to its element and one or more references to neighboring nodes in order to collectively represent the linear order of the sequence.

We will demonstrate a trade-off of advantages and disadvantages when contrasting array-based sequences and **linked** lists. Elements of a **linked** list cannot be efficiently accessed by a numeric index k , and we cannot tell just by examining a node if it is the second, i th, or twentieth node in the list. However, **linked** lists avoid the three disadvantages noted above for array-based sequences.

7.1

Singly **Linked** Lists

A singly **linked** list, in its simplest form, is a collection of nodes that collectively form a linear sequence. Each node stores a reference to an object that is an element of the sequence, as well as a reference to the next node of the list (see Figures 7.1 and 7.2).

```
next
```

```
MSP
```

```
element
```

Figure 7.1: Example of a node instance that forms part of a singly **linked** list. The node's `element` member references an arbitrary object that is an element of the sequence (the airport code MSP, in this example), while the `next` member references the subsequent node of the **linked** list (or `None` if there is no further node).

7.1. Singly Linked Lists

257

tail

MSP

BOS

ATL

head

LAX

Figure 7.2: Example of a singly linked list whose elements are strings indicating airport codes. The list instance maintains a member named head that identifies the first node of the list, and in some applications another member named tail that identifies the last node of the list. The None object is denoted as $\emptyset$.

The first and last node of a linked list are known as the head and tail of the list, respectively. By starting at the head, and moving from one node to another by following each node's next reference, we can reach the tail of the list. We can identify the tail as the node having None as its next reference. This process is commonly known as traversing the linked list. Because the next reference of a node can be viewed as a link or pointer to another node, the process of traversing a list is also known as link hopping or pointer hopping.

A linked list's representation in memory relies on the collaboration of many objects. Each node is represented as a unique object, with that instance storing a reference to its element and a reference to the next node (or None). Another object represents the linked list as a whole. Minimally, the linked list instance must keep a reference to the head of the list. Without an explicit reference to the head, there would be no way to locate that node (or indirectly, any others). There is not an absolute need to store a direct reference to the tail of the list, as it could otherwise be located by starting at the head and traversing the rest of the list. However, storing an explicit reference to the tail node is a common convenience to avoid such a traversal. In similar regard, it is common for the linked list instance to keep a count of the total number of nodes that comprise the list (commonly described as the size of the list), to avoid the need to traverse the list to count the nodes.

For the remainder of this chapter, we continue to illustrate nodes as objects, and each node's next reference as a pointer. However, for the sake of simplicity, we illustrate a node's element embedded directly within the node structure, even though the element is, in fact, an independent object. For example, Figure 7.3 is a more compact illustration of the linked list from Figure 7.2.

LAX

MSP

BOS

ATL

head

tail

Figure 7.3: A compact illustration of a singly linked list, with elements embedded in the nodes (rather than more accurately drawn as references to external objects).

Doubly Linked Lists

In a singly linked list, each node maintains a reference to the node that is immediately after it. We have demonstrated the usefulness of such a representation when managing a sequence of elements. However, there are limitations that stem from the asymmetry of a singly linked list. In the opening of Section 7.1, we emphasized that we can efficiently insert a node at either end of a singly linked list, and can delete a node at the head of a list, but we are unable to efficiently delete a node at the tail of the list. More generally, we cannot efficiently delete an arbitrary node from an interior position of the list if only given a reference to that node, because we cannot determine the node that immediately precedes the node to be deleted (yet, that node needs to have its next reference updated).

To provide greater symmetry, we define a linked list in which each node keeps an explicit reference to the node before it and a reference to the node after it. Such a structure is known as a doubly linked list. These lists allow a greater variety of $O(1)$ -time update operations, including insertions and deletions at arbitrary positions within the list. We continue to use the term `·next·` for the reference to the node that follows another, and we introduce the term `·prev·` for the reference to the node that precedes it.

Header and Trailer Sentinels

In order to avoid some special cases when operating near the boundaries of a doubly linked list, it helps to add special nodes at both ends of the list: a header node at the beginning of the list, and a trailer node at the end of the list. These `·dummy·` nodes are known as sentinels (or guards), and they do not store elements of the primary sequence. A doubly linked list with such sentinels is shown in Figure 7.10.

SFO

JFK

PVD

next

next

next

prev

prev

prev

prev

header

trailer

next

Figure 7.10: A doubly linked list representing the sequence { JFK, PVD, SFO }, using sentinels header and trailer to demarcate the ends of the list.

When using sentinel nodes, an empty list is initialized so that the next `·eld` of the header points to the trailer, and the prev `·eld` of the trailer points to the header; the remaining `·elds` of the sentinels are irrelevant (presumably `None`, in Python). For a nonempty list, the header's next will refer to a node containing the `·rst` real element of a sequence, just as the trailer's prev references the node containing the last element of a sequence.

Chapter 7. **Linked** Lists

As another example, a text document can be viewed as a long sequence of characters. A word processor uses the abstraction of a cursor to describe a position within the document without explicit use of an integer index, allowing operations such as ·delete the character at the cursor· or ·insert a new character just after the cursor·. Furthermore, we may be able to refer to an inherent position within a document, such as the beginning of a particular section, without relying on a character index (or even a section number) that may change as the document evolves.

A Node Reference as a Position?

One of the great benefits of a **linked** list **structure** is that it is possible to perform $O(1)$ -time insertions and deletions at arbitrary positions of the list, as long as we are given a reference to a relevant node of the list. It is therefore very tempting to develop an ADT in which a node reference serves as the mechanism for describing a position. In fact, our

Doubly**Linked**Base class of Section 7.3.1 has methods

insert between and delete node that accept node references as parameters.

However, such direct use of nodes would violate the object-oriented design principles of abstraction and encapsulation that were introduced in Chapter 2. There are several reasons to prefer that we encapsulate the nodes of a **linked** list, for both our sake and for the benefit of users of our abstraction.

- It will be simpler for users of our data **structure** if they are not bothered with unnecessary details of our implementation, such as low-level manipulation of nodes, or our reliance on the use of sentinel nodes. Notice that to use the insert between method of our Doubly**Linked**Base class to add a node at the beginning of a sequence, the header sentinel must be sent as a parameter.
- We can provide a more robust data **structure** if we do not permit users to directly access or manipulate the nodes. In that way, we ensure that users cannot invalidate the consistency of a list by mismanaging the linking of nodes.

A more subtle problem arises if a user were allowed to call the insert between or delete node method of our Doubly**Linked**Base class, sending a node that does not belong to the given list as a parameter. (Go back and look at that code and see why it causes a problem!)

- By better encapsulating the internal details of our implementation, we have greater ·exibility to redesign the data **structure** and improve its performance. In fact, with a well-designed abstraction, we can provide a notion of a non-numeric position, even if using an array-based sequence.

For these reasons, instead of relying directly on nodes, we introduce an independent position abstraction to denote the location of an element within a list, and then a complete positional list ADT that can encapsulate a doubly **linked** list (or even an array-based sequence; see Exercise P-7.46).

Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-7.1 Give an algorithm for finding the second-to-last node in a singly **linked** list in which the last node is indicated by a next reference of None.

R-7.2 Describe a good algorithm for concatenating two singly **linked** lists L and M, given only references to the first node of each list, into a single list L that contains all the nodes of L followed by all the nodes of M.

R-7.3 Describe a recursive algorithm that counts the number of nodes in a singly **linked** list.

R-7.4 Describe in detail how to swap two nodes x and y (and not just their contents) in a singly **linked** list L given references only to x and y. Repeat this exercise for the case when L is a doubly **linked** list. Which algorithm takes more time?

R-7.5 Implement a function that counts the number of nodes in a circularly **linked** list.

R-7.6 Suppose that x and y are references to nodes of circularly **linked** lists, although not necessarily the same list. Describe a fast algorithm for telling if x and y belong to the same list.

R-7.7 Our CircularQueue class of Section 7.2.2 provides a rotate() method that has semantics equivalent to Q.enqueue(Q.dequeue()), for a nonempty queue.

Implement such a method for the **Linked**Queue class of Section 7.1.2 without the creation of any new nodes.

R-7.8 Describe a nonrecursive method for finding, by link hopping, the middle node of a doubly **linked** list with header and trailer sentinels. In the case of an even number of nodes, report the node slightly left of center as the middle. (Note: This method must only use link hopping; it cannot use a counter.) What is the running time of this method?

R-7.9 Give a fast algorithm for concatenating two doubly **linked** lists L and M, with header and trailer sentinel nodes, into a single list L.

R-7.10 There seems to be some redundancy in the repertoire of the positional list ADT, as the operation L.add first(e) could be enacted by the alternative L.add before(L.first(), e). Likewise, L.add last(e) might be performed as L.add after(L.last(), e). Explain why the methods add first and add last are necessary.